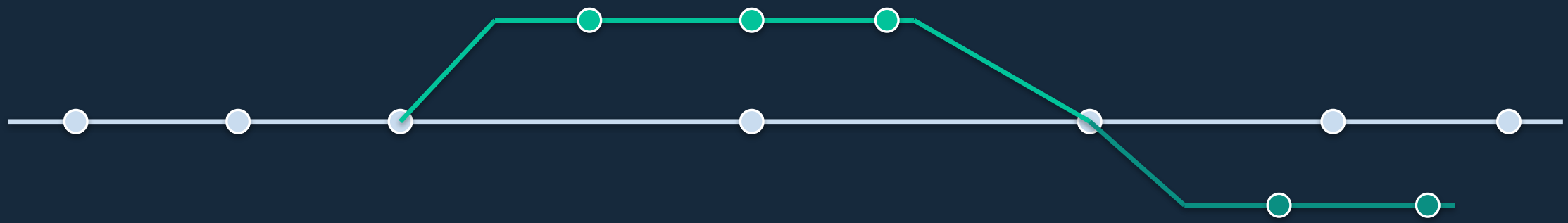
PLATFORM ENGINEERING • TEAM DISCUSSION

Release Management Architecture

Git branching & configuration management — future state for our global Java platform

US · Hong Kong · London Equities · Fixed Income · Swaps QA · UAT · Prod



THE PROBLEM

Cherry-picking is a symptom, not the disease

Today, one Git branch encodes three unrelated dimensions at once:

1. **What the code is** — a version of business logic
2. **Where it runs** — QA vs UAT vs Prod · US vs HK vs London
3. **How it's configured** — environment- & region-specific values

When environments live on branches, every fix must be manually replayed — cherry-picked — onto every branch. The fix: give each dimension its own home.

Where each dimension should live

Code version
Git history + an immutable versioned artifact in Artifactory

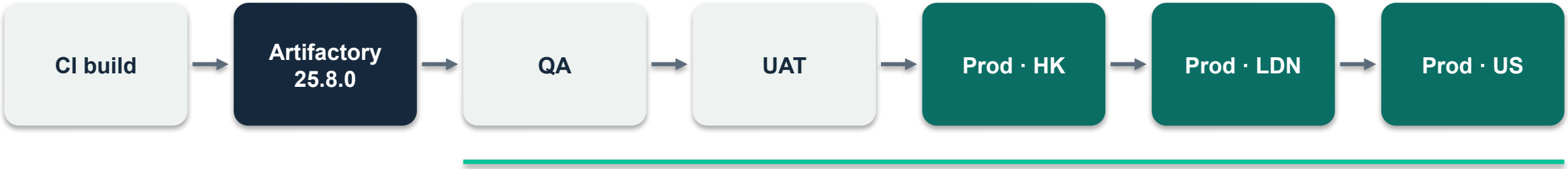
Environment / region
Directories + deployment manifests — never branches

Configuration values
Trunk-only config repo with layered overrides

Golden rule: *an environment is a place you promote an artifact to — not a branch you merge code into.*

THE SPINE OF THE PIPELINE

Build once, promote everywhere



the same bytes — the artifact tested in QA is byte-identical in every Prod region

One artifact, zero config inside

CI builds one environment-agnostic tarball per service per version. Published to Artifactory, immutable, never rebuilt.

Config rendered at deploy

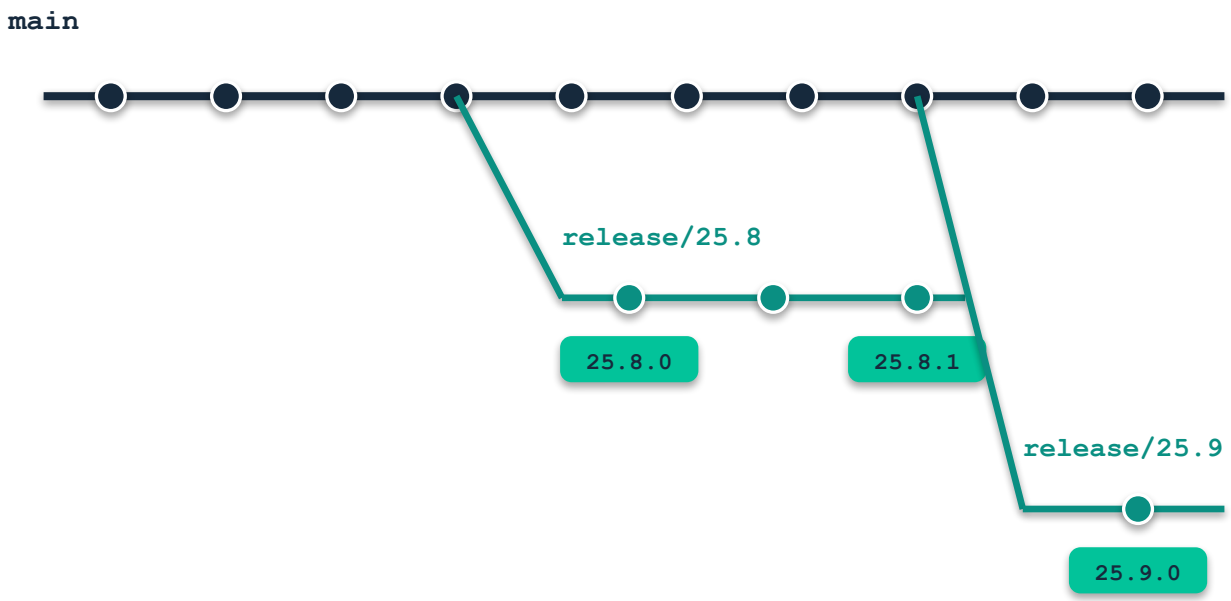
What changes per target is only the config placed next to the binary at deploy time — from the config repo.

Promotion is metadata

Moving 25.8.0 from UAT to Prod = a one-line manifest PR + an Artifactory repo move. Rebuilding for Prod = testing one binary, shipping another.

JAVA REPOSITORY

Trunk-based development + release trains



Every build that can reach Prod is a tag producing an immutable artifact.

main is the only long-lived branch.
Features merge via short-lived branches (days) with PR review + CI.

Release trains on a fixed cadence.
Cut release/25.8 every 2–4 weeks; the branch exists only for stabilization.

Tags define what ships.
25.8.0, 25.8.1 — tags, not branches, mark releases.

Hotfix = fix on main, one cherry-pick to the live release.
The only sanctioned cherry-pick in the whole system, always main → release.

GROUND RULES

What we stop doing — what we start doing

STOP

- ✕ Environment & region branches (qa, uat, prod, hk, ldn)
- ✕ Cherry-picking fixes across release / environment branches
- ✕ Rebuilding the binary per environment
- ✕ Baking configuration into the tarball
- ✕ Git Flow's develop/main split — extra merges, zero benefit

START

- ✓ Trunk + release trains; tags define every release
- ✓ One immutable artifact promoted QA → UAT → Prod
- ✓ Promotion & rollback as small, audited manifest PRs
- ✓ Config rendered at deploy time from the config repo
- ✓ Feature flags (kept few) to decouple go-live by region

*Regional go-live differences (HK enables a flow before US) become **flags + staggered manifest PRs** — never code divergence.*

BUSINESS FLOWS

One repo, many independently deployable services

```
trading-platform/              (one Git repo)
├─ libs/                       shared libraries
│  ├─ core-domain/
│  └─ messaging/
├─ services/                   one deployable each
│  ├─ mo-booking/
│  ├─ equities-trade-report/
│  ├─ fi-trade-report/
│  └─ swaps-lifecycle/
└─ build.gradle                multi-module build
```

Own artifact, own cadence. Each services/* module ships its own versioned artifact — equities can release weekly while swaps releases monthly, from the same repo.

Build only what changed. CI builds and tests affected modules per commit (Gradle build cache / path filters).

CODEOWNERS per domain. Equities owns services/equities-*, so global teams don’t trample each other.

Don’t split repos early. Split a module out later only if a domain truly needs independent tooling — early splits turn compile-time errors into cross-repo version hell.

REALITY CHECK

Adapting for human QA/UAT (few automated tests)

Pure trunk-based development assumes automated tests gate every merge. Without them, main is never “always releasable” — human QA needs a frozen target. Two changes:

1. Release branches live longer

- Cut release/25.8, build RC1 — humans test that exact artifact in QA, then UAT
- Branch lives for the whole manual cycle (2–4 weeks is normal); main keeps moving
- Hard limit: at most 1–2 active release branches, or the merge matrix returns

2. Fix direction reverses

- QA bug → fix on the release branch → new RC → humans retest just that area
- main can’t go first: it holds days of unverified work that must not ride along
- Merge — not cherry-pick — the release branch back into main; a merge can’t silently miss a fix

Transitional, not permanent: *each cycle, automate whatever QA actually caught — the highest-value tests you can write. As the suite grows, stabilization shrinks from 4 weeks toward days and the model converges on standard trunk + short release branches.*

FEATURE BACK-OUT

Throwaway integration branches (dev-1 → dev-2)

Why not revert the merge? git revert -m 1 removes the content but leaves the merge in history — when the fixed feature is re-merged later, Git sees it as “already merged” and silently brings in nothing. Rebuilding dev-N avoids the trap entirely.

- dev-N is a build artifact, not a source of truth — generated, never merged into anything, nobody commits to it
- Back-out = delete one line in the manifest; CI rebuilds dev-N+1 from the survivors
- Fixes land on the feature branch, never on dev-N (it’s disposable)
- Ship: merge accepted branches to main, tag — then verify git diff dev-3 <tag> is empty, so QA’s sign-off carries over without retesting

The candidate manifest drives everything

```
# release-candidates/25.8.yaml
integration: dev-3      # bump each rebuild
base: main@1f9c2e7     # pinned
features:
  - feature/eq-report-mifid
  - feature/mo-booking-amend
  - feature/swaps-csa-calc
  - feature/fi-curve-bootstrap
# - feature/eq-alloc-rework
#   ↑ backed out: one line,
#     CI rebuilds dev-4 + new RC
```

Conflicts: keep feature branches rebased on main; git rerere replays recorded resolutions on every rebuild.

FEATURE BACK-OUT, PART 2

Ephemeral environments — complementary, not an alternative

Throwaway integration branch

adopt now

- ✓ Pure Git process — works today with tarballs and human QA
- ✓ Makes back-out fast: one line, one rebuild
- ✓ Answers: “do these features work together?”

Ephemeral env per feature branch

adopt after containerization

- ✓ Each feature signed off in isolation before it enters the manifest — makes back-out rare
- ✓ Heavy with tarballs (VM pool, per-env DB, ports); nearly free on EKS: namespace per branch, Argo CD ApplicationSet, TTL teardown
- ✓ Answers: “does this feature work?”

End state: *the ephemeral env catches weak features before integration; dev-N catches cross-feature interactions — and either failing has a one-line back-out.*

CONFIGURATION REPOSITORY

One repo, one branch — directories for everything

```
platform-config/          (trunk-only)
├─ global/defaults.yaml
├─ services/
│  └─ equities-trade-report/
│     ├─ base.yaml
│     └─ overlays/
│        ├─ qa/  us hk ldn .yaml
│        ├─ uat/ …
│        └─ prod/ us hk ldn .yaml
├─ flags/              feature flags
└─ deploy/             release manifests →
```

HEAD of main = the world. Any historical commit is a full snapshot of every env & region — a regulator-ready audit trail.

Layered overlays, deltas only. Effective config = global ⊕ service base ⊕ env/region. Overlay files hold only differences — no drift through copies.

A config change is a PR. CI renders all 9 env×region combos + schema check; CODEOWNERS require prod approvers on overlays/prod/** and deploy/**.

No secrets in Git — ever. Config holds references (vault:secret/…); values live in Vault, later AWS Secrets Manager.

DEPLOYMENT AS DATA

Release manifests — GitOps before Kubernetes

```
# deploy/equities-trade-report/
#         prod-hk.yaml
service: equities-trade-report
version: 25.8.0    # Artifactory
configRef: 9f3ac21  # config sha
```

One tiny file per service per env×region declares what runs where.

Deploy = a PR that edits this file. Merge triggers the deploy job for exactly that env/region.

Rollback = git revert. No rebuild, no cherry-pick — seconds to execute, fully audited.

“What runs in Prod-HK right now?” Read one file. One command, always true.

Follow-the-sun rollout. Promote 25.8.0 to HK in its window, watch a trading day, then LDN, then US — three small PRs, not three branches.

This is exactly the Argo CD model. Ansible/Jenkins executes it today; on EKS, Argo CD takes over the same repo — the executor changes, the process doesn’t.

TARBALL WORLD, TODAY

Deploy-time composition

1

Pull artifact

equities-trade-report-25.8.0.tar.gz from Artifactory

2

Checkout config

platform-config at the manifest's configRef sha

3

Render config

merge layers for env=prod, region=hk

4

Fetch secrets

resolve vault:… references from Vault

5

Assemble dir

app/ + config/ + secrets — side by side

6

Swap + verify

stop, symlink swap, start, health check

The binary never changes per environment — only what sits next to it does.

Executed by Ansible (or a Jenkins deploy job), triggered by the manifest PR merge.

The same six steps become the container entrypoint later — image = step 1; ConfigMaps/Secrets = steps 2–4.

CI/CD STACK

Tooling — on-prem today, AWS tomorrow

Layer	On-prem today	AWS tomorrow
Source control	GitHub Enterprise / GitLab / Bitbucket — branch protection + CODEOWNERS required	same
CI (build & test)	GitLab CI or GitHub Actions self-hosted runners (Jenkins OK if entrenched)	identical — runners move to AWS
Artifacts	JFrog Artifactory (or Nexus) — tarballs now; Docker images & Helm charts later	same instance or SaaS
Secrets	HashiCorp Vault	AWS Secrets Manager (or keep Vault)
CD executor	Ansible / Jenkins deploy jobs, driven by manifest merges	EC2: CodeDeploy · EKS: Argo CD
Deploy state	platform-config repo, deploy/ directory	same repo — becomes Argo CD's source of truth

Only the executor row changes across the cloud migration — *pipelines, artifacts, config, and process carry over unchanged.*

PATH TO THE CLOUD

Tarball → containers: the tarball isn't the problem

Config inside the tarball is the problem. Fix that first, and a Docker image is just a tarball with better tooling.

NOW

Strip config out

Tarball = code only; config rendered at deploy from platform-config. Process change, no new infra.

NOW+

Build images in parallel

CI also builds a Docker image from the same jars (Jib / Boot buildpacks — no Dockerfile). Nobody deploys it yet.

EC2

Deploy the image

compose/systemd on EC2; config mounted from rendered files / SSM.

EKS

Same process, new executor

Overlays → Kustomize/ConfigMaps 1:1; deploy/ → Argo CD apps; External Secrets → Vault/ASM.

Invariant at every step: *image = code only · config injected at deploy · promotion = retag / manifest edit — never rebuild for an environment.*

Bonus at EKS: ephemeral per-feature environments (slide 9) become nearly free — namespace per branch, Argo CD ApplicationSet, TTL teardown.

PROPOSED SEQUENCE

Migration order — each step pays off on its own

1. **Extract config** Move config out of the Java repo into platform-config (trunk-only, layered dirs). Delete config from tarballs; render at deploy. Kills most cherry-picking by itself.

2. **Immutable artifacts + trunk** Artifactory promotion, versioned tarballs. Kill environment branches; adopt trunk + release trains (long stabilization while QA is manual).

3. **Release manifests** Add deploy/ manifests; Ansible/Jenkins deploys only what manifests declare. Rollback = git revert. Throwaway dev-N integration branches for back-out.

4. **Images in parallel** CI builds Jib images from the same jars alongside tarballs — unused in prod, zero risk.

5. **AWS** EC2 first with the same manifests, then EKS + Argo CD + Kustomize — same config repo, same process, new executor. Ephemeral envs become cheap.

FOR DISCUSSION

- Train cadence per desk — weekly? monthly?
- Who approves overlays/prod/** and deploy/** PRs?
- First candidate service for step 1?
- Where do we start writing the first automated tests?

Full detail: release-management-architecture.md in the ci-cd-design vault